Advanced BPF profiling techniques

How to escape NMI curse and fetch ELF contents in BPF

Andrii Nakryiko

Meta



東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DEC. 11-13, 2025

Basic BPF-based profiler ingredients

Trigger event	**BPF handler**	**Post-processing**
(e.g., cpu-cycles)	SEC("perf_event")	data collection symbolization aggregation

```
$ cat profile.{c,bpf.c} | wc -l
323
```

https://github.com/libbpf/libbpf-bootstrap

BPF profiler 101: **trigger event**

```
int num_cpus = libbpf_num_possible_cpus();
for (int cpu = 0; cpu < num_cpus; cpu++) {
    struct perf_event_attr attr;
    memset(&attr, 0, sizeof(attr));
    attr.size = sizeof(attr);

    attr.type = PERF_TYPE_HARDWARE;
    attr.config = PERF_COUNT_HW_CPU_CYCLES;
    attr.sample_period = 5000000;

    int pefd = sys_perf_event_open(&attr, -1, cpu, -1, 0);

    int link_fd = bpf_program__attach_perf_event(skel->progs.profile, pefd);
}
```

BPF profiler 101: **BPF logic**

```
SEC("perf_event")
int profile(struct pt_regs *ctx)
{
    struct event *e = bpf_ringbuf_reserve(&events, sizeof(*e), 0);

    e->pid = bpf_get_current_pid_tgid() >> 32;
    e->cpu_id = bpf_get_smp_processor_id();
    bpf_get_current_comm(e->comm, sizeof(e->comm));

    e->kstack_sz = bpf_get_stack(ctx, e->kstack, sizeof(e->kstack), 0);
    e->ustack_sz = bpf_get_stack(ctx, e->ustack, sizeof(e->ustack),
                                 BPF_F_USER_STACK);
    bpf_ringbuf_submit(e, 0);
    return 0;
}
```

BPF profiler 101: **data collection**

```
int rb_fd = bpf_map__fd(skel->maps.events);
struct ring_buffer *rb = ring_buffer__new(rb_fd, event_handler, NULL, NULL);
while (ring_buffer__poll(rb, -1) >= 0) { /* no-op */ }
...
static int event_handler(void *ctx, void *data, size_t size)
{
        struct stacktrace_event *e = data;

        printf("PID %d (%s) @ CPU %d:\n", e->pid, e->comm, e->cpu);
        show_stack_trace(event->kstack, event->kstack_sz, 0);
        show_stack_trace(event->ustack, event->ustack_sz, event->pid);

        return 0;
}
```

BPF profiler 101: **symbolization**

```
#include "blazesym.h"

struct blaze_symbolizer *S = blaze_symbolizer_new();

static void show_stack_trace(__u64 *stack, int stack_sz, pid_t pid)
{
    const struct blaze_syms *syms;
    if (pid) {
        struct blaze_symbolize_src_process src = { ..., .pid = pid };
        syms = blaze_symbolize_process_abs_addrs(S, &src, stack, stack_sz);
    } else {
        struct blaze_symbolize_src_kernel src = { ... };
        syms = blaze_symbolize_kernel_abs_addrs(S, &src, stack, stack_sz);
    }
    ...
```

BPF profiler 101: **symbolization (continued)**

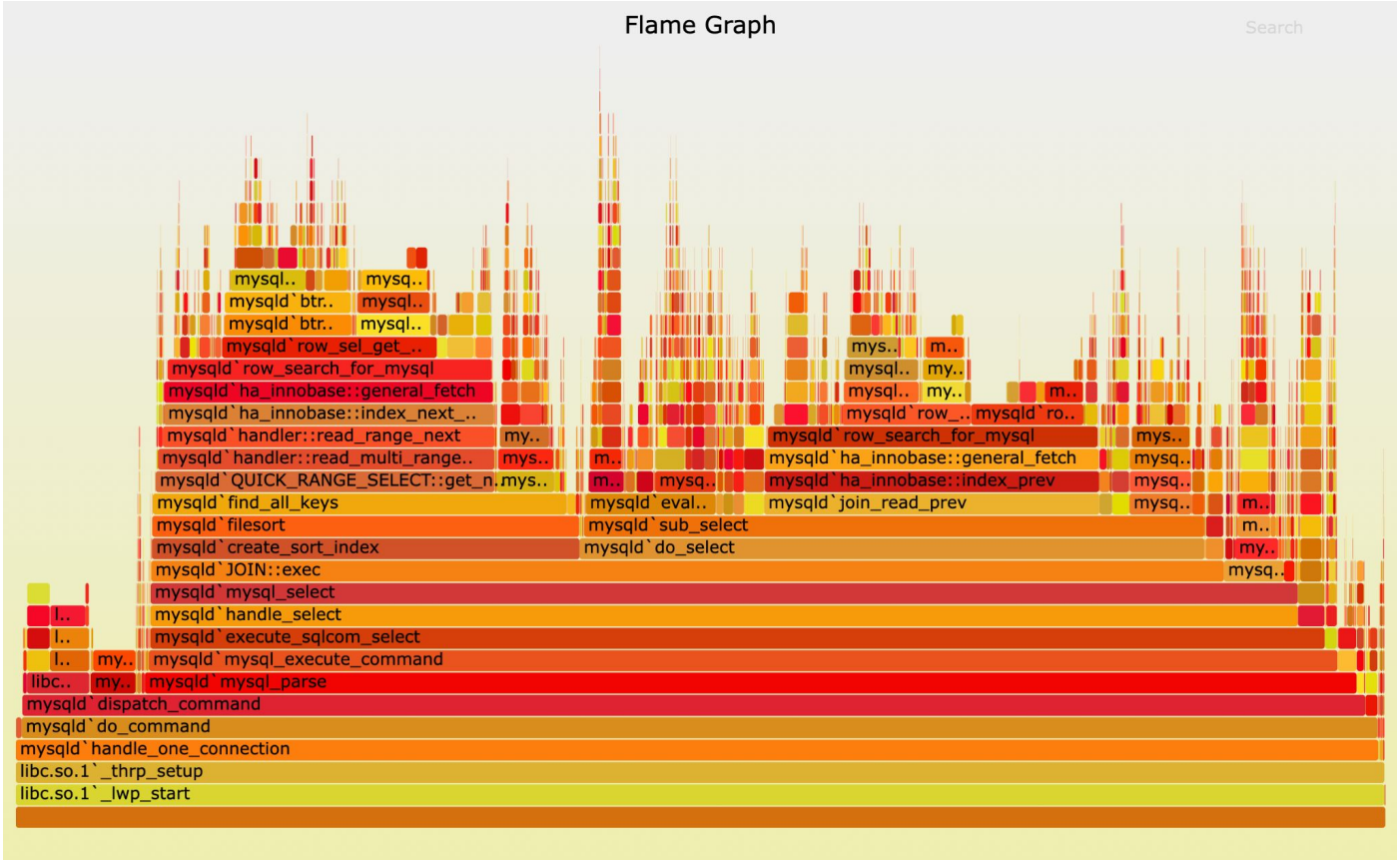
```
    ...

    for (int i = 0; i < stack_sz; i++) {
            const struct blaze_sym *sym = &syms->syms[i];
            if (!syms || !syms->syms[i].name)
                printf("%llx (<no-symbol>)\n", stack[i]);
            else
                printf("%s+%lld\n", sym->name, sym->offset);
            }
    }

    blaze_syms_free(syms);
}
```

https://github.com/libbpf/blazesym

BPF profiler 101: **aggregation**



https://www.brendangregg.com/flamegraphs.html

- libbpf-bootstrap is a good playground
  - C: examples/c/profile.{c, bpf.c, h}
  - Rust: examples/rust/profile
- blazesym saves **LOTS** of time and effort

- trigger event defines **quantifiable resource**
  - hardware events (CPU cycles, LLC misses, etc)
  - context switch, wakeup event
  - syscall, API calls
- stack trace answers **where in the code**
  - **function** causing the use of the resource
  - how we arrived at that function (**call chain**)

Application-specific context augmentation

- stack traces need **application context**
  - RPC request info (ID, name, parameters)
  - app state (e.g., forward/backward AI training pass)
  - application-specific metadata
- **extra processing** for runtimes (Python, Java, etc.)
  - Python presence and version detection
  - Python stack trace capture

Application-specific context: current solution

- stack traces need **application context**
  - **solved with thread-local storage (TLS)**
  - *... and abstractions built on top of TLS*
  - decouples profiler from specific application
- **extra processing** for runtimes (**Python**, Java, etc.)
  - solved using global and thread-local variables
  - and ***lots*** of version-specific logic (see PyStacks lib)

Data discovery problem

- **where** and **what** to read from memory
  - each running process is checked (using **procfs**)
  - **ELF parsing** is involved (ELF symbols, relocations)
  - **expensive upfront cost**
    - many processes are irrelevant in practice
    - but you can't know upfront which ones
  - **racy**
    - process can start after discovery step

Data capture problem: **the curse of NMI**

- tracing (perf/kprobe/fentry/tracepoint) is **non-sleepable**
  - worst case runs in **NMI context**
- BPF cannot cause page fault to wait for data
- unreliable user space memory reads
  - if memory is paged out, -EFAULT
  - untouched file-backed memory (*string constants*), -EFAULT
- timer and workqueue don't work in NMI mode (*we are fixing it*)
- unreliable memory allocations (*kmalloc_nolock* should help)
- unreliable locking (hashmap, ringbuf can *randomly fail*)

Solution: postpone with task work facility

- kernel-internal task_work_add() API
- asynchronous callback-based API approach
- code called **right before return to user space**
- **sleepable mode** (page faults are allowed)
- *can be made to work* from NMI mode (irq_work)

bpf: Introduce deferred task context execution by Mykyta Yatsenko

Escaping the NMI: task work

- capture kernel-side stack trace ***synchronously***
- capture user space data ***asynchronously*** in callback
- userspace stays suspended the entire time
  - same stack trace
  - same thread-local state and data
- join in BPF map or in user space post-processing

Escaping the NMI: task work API

```
struct bpf_task_work {
        __u64 __opaque;
};

int bpf_task_work_schedule_resume(
        struct task_struct *task,
        struct bpf_task_work *tw, void *map__map,
        bpf_task_work_callback_t callback) __ksym;


typedef int (*bpf_task_work_callback_t)(struct bpf_map *map,
                                        void *key, void *value);
```

(well, it's actually more like this right now)

```
struct bpf_task_work {
        __u64 __opaque;
};

int bpf_task_work_schedule_resume_impl(
        struct task_struct *task,
        struct bpf_task_work *tw, void *map__map,
        bpf_task_work_callback_t callback,
        void *aux__prog) __ksym;

typedef int (*bpf_task_work_callback_t)(struct bpf_map *map,
                                        void *key, void *value);
```

Escaping the NMI: task work example usage

```
struct task_state {
        struct stacktrace_event event;
        struct bpf_task_work tw;
        ... any needed context info here ...
};

struct {
        __uint(type, BPF_MAP_TYPE_HASH);
        __uint(max_entries, 4096);
        __type(key, u32);
        __type(value, struct task_state);
} task_states SEC(".maps");
```

Escaping the NMI: task work example usage

```
state = bpf_map_lookup_elem(&task_states, &pid);

bpf_task_work_schedule_resume_impl(
         task,
         &state->tw, &task_states,
         task_work_cb,
         NULL);

static int task_work_cb(struct bpf_map *map, void *key, void *value)
{
        struct task_state *state = value;
        … we are in sleepable context …
        bpf_map_delete_elem(&task_states, key);
        return 0;
}
```

(well, there are two flavors, actually)

```
int bpf_task_work_schedule_resumesignal(
        struct task_struct *task,
        struct bpf_task_work *tw, void *map__map,
        bpf_task_work_callback_t callback) __ksym;
```

Wall-clock (real time) profiling

```
struct task_struct *t;
bpf_for_each(task, t, 0) {
    /* get information about task t */
    /* send it over BPF ringbuf */
}
```

- quantify actual time spent by thread
- thread can be blocked, in I/O, or even *deadlocked!*
- you'll still observe what it's doing and where

Problem: unreliable stack traces

- task might be running in parallel on another CPU
- bpf_get_stack() *will try capturing* kernel stack trace
  - but unreliable: **will bail** if task is running on another CPU
- bpf_get_stack() **won't even try** capturing user stack trace
  - needs to be implemented in BPF logic manually
  - *very unreliable*, can change any time

We have to suspend the task (briefly).

Wall-clock profiling: stack trace capture

```
int bpf_task_work_schedule_signal(
        struct task_struct *task,
        struct bpf_task_work *tw, void *map__map,
        bpf_task_work_callback_t callback) __ksym;
```

- *callback* will run with **task as current**
- if task sleeps, it's woken up briefly
- if task runs, it's interrupted briefly

https://github.com/libbpf/libbpf-bootstrap/blob/lpc-task-work-file-dynptr/examples/c/snooper.bpf.c

Wall-clock profiling: stack trace capture

```
regs = bpf_core_cast((void *)bpf_task_pt_regs(task), struct pt_regs);
stack[0] = regs->ip;
fp = regs->bp;
bpf_for(i, 1, MAX_STACK_DEPTH) {
    /* read the frame, [fp] = next_fp, [fp+8] = ret_addr */
    if (bpf_copy_from_user_task(&frame, sizeof(frame), (void *)fp, task, 0))
            break;

    barrier_var(i);
    if (i < MAX_STACK_DEPTH)
        stack[i] = frame.ret_addr;

    fp = frame.next_fp;
}
```

https://github.com/libbpf/libbpf-bootstrap/blob/lpc-task-work-file-dynptr/examples/c/snooper.bpf.c

Data discovery problem

- **where** and **what** to read from memory
  - each running process is checked (using **procfs**)
  - **ELF parsing** is involved (ELF symbols, relocations)
  - **expensive upfront cost**
    - many processes are irrelevant in practice
    - but you can't know upfront which ones
  - **racy**
    - process can start after discovery step

Data discovery problem

- **where** and **what** to read from memory
  - each running process is checked (using **procfs**)
  - **ELF parsing** is involved (ELF symbols, relocations)
  - **expensive upfront cost**
    - many procedures are irrelevant in practice
    - but you can't know upfront which ones
  - **racy**
    - process can start after discovery step

Can we do better with BPF?

How to read ELF contents from BPF?

How to read ELF contents from BPF?

We actually solved this problem already!

How to read ELF contents from BPF?

- bpf_get_stack() can fetch build ID
- build ID is in (*non-allocatable*) **.note** section of ELF
  - not memory-mapped, *can't bpf_probe_read it*
  - has to read it from file (*even in NMI context*)
- **file reader abstraction** (`struct freader` in lib/buildid.c)
  - holds at most one folio (page) at a time
  - uses temporary buffer for cross-page reads

How do we expose file reader to BPF?

- **BPF dynptr** abstraction
  - logical range of sequential N bytes
  - **bpf_dynptr_read()** for generic reading into buffer
    - takes offset and size
    - no static (verifier) restrictions
    - but destination buffer has to be sized statically
      (maximum read is bounded by buffer size)

File is *exactly* a **logically consecutive** N bytes!

BPF file-based dynptr: API

```
struct bpf_dynptr; /* 16-byte opaque struct */

int bpf_dynptr_from_file(struct file *file, u32 flags,
                         struct bpf_dynptr *ptr__uninit) __ksym;

int bpf_dynptr_file_discard(struct bpf_dynptr *dynptr) __ksym;

long bpf_dynptr_read(void *dst, u64 len,
                     const struct bpf_dynptr *src,
                     u64 offset, u64 flags);
```

bpf: Introduce file dynptr by Mykyta Yatsenko

Source of files: task's VMA iterator

```
struct task_struct *task = ...;
struct vm_area_struct *vma;
bpf_for_each(task_vma, vma, task, 0) {
        if (!(vma->vm_flags & VM_EXEC))
                continue;
        struct *file = vma->vm_file;
        if (!file)
                continue;
        struct bpf_dynptr fdptr;
        bpf_dynptr_from_file(file, 0, &fdptr);
        ... ELF parsing here using bpf_dynptr_read(&fdptr) ...
        bpf_dynptr_file_discard(&fdptr);
}
```

BPF file-based dynptr: ELF parsing example

- libbpf-bootstrap "snooper" demo (lpc-task-work-file-dynptr branch)
- **SEC("iter/task")** task iterator program
  - alternatively use **bpf_for_each(task)** open-coded iterator approach
- **bpf_task_work_schedule_signal()** for targeted tasks to suspend
  - capture user stack trace (manual frame pointer unrolling loop)
- iterate all file-backed executable VMAs with **bpf_for_each(task_vma)**
  - for each executable search for **PyVersion** ELF symbol and read it *reliably*
    - a simple Python presence and **version detection**

But wait, there is more!

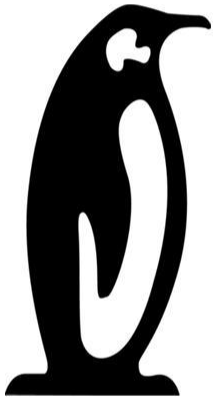
- proof-of-concept **thread-local variable reading**
  - **on-demand and purely from BPF**
  - using **ELF symbol and ELF relocations**
  - determine TLS module ID and variable offset
  - read it for current task (using **task->thread.fsbase**)
  - supports local exec and general dynamic TLS modes
- highly recommended reads
  - Ulrich Drepper's ELF Handling For Thread-Local Storage paper
  - A Deep dive into (implicit) Thread Local Storage (very approachable)
  - All about thread-local storage (exhaustive)

Summary

- Basic BPF profiler is simple
  - profiler example in libbpf-bootstrap is a simple starting point
  - use blazesym for symbolization, avoid this daunting task
- Use **bpf_task_work_schedule_{resume,signal}()** to get into sleepable mode
- Parsing **ELF** (and **DWARF**!) is now doable **purely from BPF**
  - task_vma iterator + **bpf_dynptr_from_file()** + bpf_dynptr_read()
  - eliminate expensive and inconvenient user space pre-processing
- Check the demo for inspiration
- **Let's build solid BPF-side libraries!**
  - ELF parsing (libelf for BPF?)
  - thread-local storage (TLS) reading

Thank you!





東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

